

Fundamentals of Data Structures

Project 1: Searching Algorithms Performance Analysis



Author:

Date: 2026/03/17

2025-2026 Spring & Summer Semester

Table of Contents

Chapter 1: Introduction	3
Chapter 2: Algorithm Specification	3
2.1. Sequential Search (Iterative)	3
2.2. Sequential Search (Recursive)	4
2.3. Binary Search (Iterative)	4
2.4. Binary Search (Recursive)	5
2.5. Testing Program (Main)	5
2.6. Summary of Complexities	6
Chapter 3: Testing Results	6
3.1. Correctness Test Cases	6
3.2. Measurement Methodology	7
3.3. Performance Results	7
3.4. Performance Plot	8
Chapter 4: Analysis and Comments	8
4.1. Time Complexity Analysis	8
4.2. Space Complexity Analysis	9
4.3. Iterative vs. Recursive	9
4.4. Binary Search vs. Sequential Search	9
4.5. Possible Improvements	9
Appendix: Source Code (in C)	10
Declaration	14

Chapter 1: Introduction

In computer science, searching is one of the most fundamental operations performed on data structures. Given a collection of N ordered integers, efficiently locating a target value (or determining its absence) is critical in many applications such as database querying, information retrieval, and data processing.

This project investigates two classical searching algorithms: **Sequential Search** and **Binary Search**. Sequential search examines elements one by one from left to right, while binary search exploits the sorted order of the data by repeatedly halving the search space. For each algorithm, we implement both an **iterative version** and a **recursive version**, yielding four distinct implementations in total.

The main data structure used is a simple **integer array** `list[0..N-1]`, storing N sorted integers from 0 to $N - 1$. No additional complex data structures are needed, since both algorithms operate directly on a contiguous sorted array via index-based access.

The goal of this project is threefold:

1. Implement the four searching functions in C and verify their correctness.
2. Analyze the worst-case time and space complexities of each version theoretically.
3. Measure and compare the actual running times for various input sizes $N = 100, 500, 1000, 2000, 4000, 6000, 8000, 10000$, using the worst-case scenario where the target value does not exist in the list.

The worst case is constructed by searching for the value N in a sorted list of integers from 0 to $N - 1$. Since N is not in the list, every algorithm must exhaust all possibilities before concluding the search, which guarantees worst-case behavior.

Chapter 2: Algorithm Specification

2.1. Sequential Search (Iterative)

The iterative sequential search scans the array from index 0 to $N - 1$. If the target is found, it returns the index; otherwise it returns -1 .

```
1 Input: array list[], size N, target value target
2 Set  $i = 0$ 
3 while  $i < N$  and  $\text{list}[i] \neq \text{target}$ 
4   |  $i = i + 1$ 
5 end
6 if  $i < N$  then
7   | return  $i$ 
8 else
9   | return  $-1$ 
```

10 **end**

Time Complexity: In the worst case, the target is not in the list, so the loop runs N times. Thus the worst-case time complexity is $O(N)$.

Space Complexity: $O(1)$, only a constant amount of extra space is used.

2.2. Sequential Search (Recursive)

The recursive version checks the current index: if it equals N , the search fails; if the current element matches the target, the index is returned; otherwise, the function calls itself with `index + 1`.

```
1 Input: array list[], size N, target value target, current index
2 if index =  $N$  then
3   | return -1
4 end
5 if list[index] = target then
6   | return index
7 end
8 return SequentialSearch_Recursive(list, N, target, index + 1)
```

Time Complexity: Worst case $O(N)$, same as the iterative version since N recursive calls are made.

Space Complexity: $O(N)$, because each recursive call adds a frame to the call stack.

2.3. Binary Search (Iterative)

The iterative binary search maintains two pointers `left` and `right`. At each step, it computes the midpoint and compares the middle element with the target, then narrows the search range by half.

```
1 Input: array list[], size N, target value target
2 Set left = 0, right =  $N - 1$ 
3 while left ≤ right
4   | mid =  $\frac{\text{left} + \text{right}}{2}$ 
5   | if list[mid] > target then
6     | right = mid - 1
7   | else if list[mid] < target then
8     | left = mid + 1
9   | else
```

```

10 | | break (found)
11 | end
12 end
13 if left ≤ right then return mid
14 else return -1

```

Time Complexity: In the worst case, the search space is halved each iteration. The loop runs at most $\lfloor \log_2 N \rfloor + 1$ times. Thus the worst-case time complexity is $O(\log N)$.

Space Complexity: $O(1)$.

2.4. Binary Search (Recursive)

The recursive version follows the same divide-and-conquer logic: check the base case, compute the midpoint, and recurse on the appropriate half.

```

1 Input: array list[], target value target, left, right
2 if left > right then
3 | return -1
4 end
5 mid =  $\frac{\text{left} + \text{right}}{2}$ 
6 if list[mid] = target then
7 | return mid
8 else if list[mid] > target then
9 | return BinarySearch_Recursive(list, target, left, mid - 1)
10 else
11 | return BinarySearch_Recursive(list, target, mid + 1, right)
12 end

```

Time Complexity: Worst case $O(\log N)$, same as the iterative version.

Space Complexity: $O(\log N)$, due to the recursion depth.

2.5. Testing Program (Main)

The main program drives the entire experiment. For each value of N , it constructs a sorted array, then measures the worst-case running time of all four search functions using `clock()` from `<time.h>`. The repetition factor K is dynamically determined to ensure measurement accuracy.

```

1 Define test sizes:  $N \in \{100, 500, 1000, 2000, 4000, 6000, 8000, 10000\}$ 
2 for each  $N$  in test sizes

```

```

3 | Allocate array  $a[0..N-1]$  and fill with  $a[i] = i$ 
4 | Set target =  $N$  (guarantees worst case: not found)
5 | for each search function  $f$ 
6 |   Set  $K = 1$ 
7 |   do
8 |      $K = K \times 2$ 
9 |     start = clock()
10 |    Repeat calling  $f$  for  $K$  times
11 |    stop = clock()
12 |    while stop - start < 10
13 |    Compute and print Ticks, Total Time, Duration
14 | end for
15 | Free array  $a$ 
16 end for

```

The key design decision is the dynamic K -doubling strategy: instead of using a fixed repetition count, the program keeps doubling K until the elapsed ticks reach at least 10. This ensures that even extremely fast functions (like binary search on small arrays) are measured with sufficient precision.

2.6. Summary of Complexities

Algorithm	Time (Worst Case)	Space
Binary Search (Iterative)	$O(\log N)$	$O(1)$
Binary Search (Recursive)	$O(\log N)$	$O(\log N)$
Sequential Search (Iterative)	$O(N)$	$O(1)$
Sequential Search (Recursive)	$O(N)$	$O(N)$

Chapter 3: Testing Results

3.1. Correctness Test Cases

Before measuring performance, we verify that all four functions produce correct results under different scenarios. The test array is `list[] = {0, 1, 2, ..., 9}` with $N = 10$.

Case ID	Test Type	Target	Purpose	Expected	Actual	Status
TC-01	Best case	5	Search for middle element (best case for binary search).	Return 5	All 4 functions return 5	pass

Case ID	Test Type	Target	Purpose	Expected	Actual	Status
TC-02	Boundary (first)	0	Search for the first element (left boundary).	Return 0	All 4 functions return 0	pass
TC-03	Boundary (last)	9	Search for the last element (right boundary).	Return 9	All 4 functions return 9	pass
TC-04	Worst case	10	Search for a value not in the list (worst case).	Return -1	All 4 functions return -1	pass
TC-05	Edge case	$N = 1$, target=0	Single-element array, target exists.	Return 0	All 4 functions return 0	pass
TC-06	Edge case	$N = 1$, target=1	Single-element array, target not found.	Return -1	All 4 functions return -1	pass
TC-07	Large N	$N = 10000$, target= N	Comprehensive worst-case test on the largest input.	Return -1	All 4 functions return -1	pass

3.2. Measurement Methodology

To measure the running time of each function, we use the C standard library `<time.h>`. The `clock()` function records the processor time in ticks before and after the function call. Since a single function call may be too fast to measure accurately, we repeat the call K times, where K is dynamically determined by doubling until the elapsed ticks are at least 10. This ensures a measurement accuracy of at least 10%.

The key formulas are:

$$\text{Ticks} = \text{stop} - \text{start}$$

$$\text{Total Time (sec)} = \frac{\text{Ticks}}{\text{CLOCKS_PER_SEC}}$$

$$\text{Duration (sec)} = \frac{\text{Total Time}}{K}$$

3.3. Performance Results

The following table shows the worst-case performance results for each algorithm under different values of N . The target value is N (not in the list), providing the worst case.

	Metric	100	500	1000	2000	4000	6000	8000	10000
Binary Search (iterative)	Iterations (K)	1048576	524288	524288	524288	524288	524288	524288	524288
	Ticks	15	11	13	13	16	17	16	20
	Total Time (sec)	0.015	0.011	0.013	0.013	0.016	0.017	0.016	0.020
	Duration (sec)	1.43e-8	2.10e-8	2.48e-8	2.48e-8	3.05e-8	3.24e-8	3.05e-8	3.81e-8

	Metric	100	500	1000	2000	4000	6000	8000	10000
Binary Search (recursive)	Iterations (K)	1048576	1048576	524288	524288	524288	524288	524288	524288
	Ticks	15	20	10	11	12	14	14	14
	Total Time (sec)	0.015	0.020	0.010	0.011	0.012	0.014	0.014	0.014
	Duration (sec)	1.43e-8	1.91e-8	1.91e-8	2.10e-8	2.29e-8	2.67e-8	2.67e-8	2.67e-8
Sequential Search (iterative)	Iterations (K)	262144	32768	32768	16384	4096	4096	4096	2048
	Ticks	18	10	17	18	10	14	17	12
	Total Time (sec)	0.018	0.010	0.017	0.018	0.010	0.014	0.017	0.012
	Duration (sec)	6.87e-8	3.05e-7	5.19e-7	1.10e-6	2.44e-6	3.42e-6	4.15e-6	5.86e-6
Sequential Search (recursive)	Iterations (K)	32768	4096	2048	1024	512	256	128	128
	Ticks	19	14	14	16	18	14	10	12
	Total Time (sec)	0.019	0.014	0.014	0.016	0.018	0.014	0.010	0.012
	Duration (sec)	5.80e-7	3.42e-6	6.84e-6	1.56e-5	3.52e-5	5.47e-5	7.81e-5	9.38e-5

3.4. Performance Plot

The following figure plots the Duration (seconds) of all four algorithms against the input size N on the same coordinate system with a logarithmic y -axis.

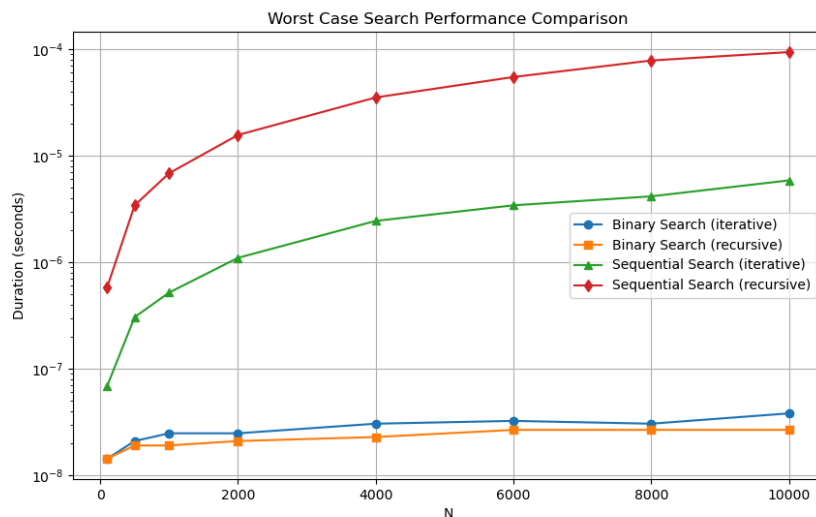


Figure 1: Comparison of worst-case running times for four searching algorithms.

Chapter 4: Analysis and Comments

4.1. Time Complexity Analysis

Binary Search (both versions): The worst-case time complexity is $O(\log N)$. Both the iterative and recursive versions have Duration values on the order of 10^{-8} seconds,

and the values barely increase as N grows from 100 to 10000. This is consistent with the theoretical analysis: $\log_2(100) \approx 6.6$ and $\log_2(10000) \approx 13.3$, so the number of comparisons only roughly doubles when N increases by a factor of 100.

Sequential Search (both versions): The worst-case time complexity is $O(N)$. The iterative version's Duration increases from approximately 6.87×10^{-8} seconds at $N = 100$ to 5.86×10^{-6} seconds at $N = 10000$, which is roughly an 85x increase for a 100x increase in N . This closely matches the predicted $O(N)$ behavior.

4.2. Space Complexity Analysis

Iterative versions: Both iterative algorithms use $O(1)$ extra space, since they only maintain a few local variables (loop counters, pointers).

Recursive versions: Binary search (recursive) uses $O(\log N)$ stack space due to $\log N$ levels of recursion. Sequential search (recursive) uses $O(N)$ stack space because, in the worst case, it makes N recursive calls. This large stack usage is a practical disadvantage and may cause stack overflow for very large N .

4.3. Iterative vs. Recursive

Although the iterative and recursive versions of each algorithm share the same time complexity, the recursive versions are noticeably slower in practice. This is most evident for sequential search: the recursive version is about 10x–16x slower than the iterative version across all N values. Each recursive call incurs overhead from the function call mechanism — pushing parameters, return addresses, and local variables onto the call stack. In contrast, the iterative version uses a simple loop with minimal overhead per iteration.

4.4. Binary Search vs. Sequential Search

At $N = 10000$, binary search (iterative) takes about 3.81×10^{-8} seconds per call, while sequential search (iterative) takes about 5.86×10^{-6} seconds — a difference of over 150x. This gap widens further as N increases, since $O(\log N)$ grows much slower than $O(N)$.

4.5. Possible Improvements

- **Sentinel optimization:** Use a sentinel in sequential search to eliminate the boundary check `i < N` in each iteration, reducing the constant factor.
- **Interpolation search:** For uniformly distributed data, interpolation search can achieve $O(\log \log N)$ average time, outperforming standard binary search.
- **Cache effects:** For very large arrays, memory access patterns may affect real-world performance beyond what asymptotic analysis predicts.

- **Binary search requires sorted data:** If data is unsorted, the cost of sorting ($O(N \log N)$) must be considered. For a single search on unsorted data, sequential search may be more practical.

Appendix: Source Code (in C)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <time.h>
4
5  /* =====
6   * Binary Search - Iterative Version
7   * Searches for 'target' in a sorted array 'list' of size N.
8   * Returns the index if found, -1 otherwise.
9   * Time Complexity: O(log N) Space Complexity: O(1)
10  * ===== */
11 int BinarySearch_Iterative(int list[], int N, int target) {
12     int left = 0, right = N - 1; /* initialize search range */
13     int mid;
14     while (left <= right) {      /* loop until range is empty */
15         mid = (left + right) / 2; /* compute midpoint */
16         if (list[mid] > target) {
17             right = mid - 1;     /* target in left half */
18         }
19         else if (list[mid] < target) {
20             left = mid + 1;      /* target in right half */
21         }
22         else {
23             break;              /* target found */
24         }
25     }
26     if (left <= right) return mid; /* found: return index */
27     else return -1;             /* not found */
28 }
29
30 /* =====
31  * Binary Search - Recursive Version
32  * Searches for 'target' in list[left..right].
33  * Returns the index if found, -1 otherwise.
34  * Time Complexity: O(log N) Space Complexity: O(log N)
35  * ===== */
36 int BinarySearch_Recursive(int list[], int target, int left, int right) {
37     if (left > right) return -1; /* base case: empty range */
38     int mid = (left + right) / 2; /* compute midpoint */
39     if (list[mid] == target) return mid; /* found */

```

```

40     else if (list[mid] > target)
41         return BinarySearch_Recursive(list, target, left, mid - 1); /* search
                                left half */
42     else
43         return BinarySearch_Recursive(list, target, mid + 1, right); /* search
                                right half */
44 }
45
46 /* =====
47 * Sequential Search - Iterative Version
48 * Scans array from left to right looking for 'target'.
49 * Returns the index if found, -1 otherwise.
50 * Time Complexity: O(N) Space Complexity: O(1)
51 * ===== */
52 int SequentialSearch_Iterative(int list[], int N, int target) {
53     int i = 0;
54     while (i < N && list[i] != target) { /* scan until found or end */
55         i++;
56     }
57     if (i < N) return i; /* found: return index */
58     else return -1; /* not found */
59 }
60
61 /* =====
62 * Sequential Search - Recursive Version
63 * Checks current index, then recurses on the next position.
64 * Returns the index if found, -1 otherwise.
65 * Time Complexity: O(N) Space Complexity: O(N)
66 * ===== */
67 int SequentialSearch_Recursive(int list[], int N, int target, int index) {
68     if (index == N) return -1; /* base case: end of array */
69     else if (list[index] == target) return index; /* found */
70     return SequentialSearch_Recursive(list, N, target, index + 1); /* check
                                next */
71 }
72
73 /* =====
74 * printResult - Helper function to print timing results
75 * Computes Total Time and Duration from ticks and K,
76 * then prints them in a formatted line.
77 * ===== */
78 void printResult(char *name, int K, long ticks) {
79     double totalTime = (double)ticks / CLOCKS_PER_SEC; /* convert ticks to
                                seconds */
80     double duration = totalTime / K; /* single-call
                                duration */

```

```

81     printf("%-35s K=%-10d Ticks=%-10ld Total Time=%-12f Duration=%e\n",
82            name, K, ticks, totalTime, duration);
83 }
84
85 /* =====
86  * main - Testing program
87  * For each N, dynamically determines K (repetition count)
88  * so that elapsed ticks >= 10 for measurement accuracy.
89  * Tests all four search functions under worst case (target = N).
90  * ===== */
91 int main() {
92     int testN[] = {100, 500, 1000, 2000, 4000, 6000, 8000, 10000};
93     int numTests = 8; /* number of different N values to test */
94
95     for (int t = 0; t < numTests; t++) {
96         int N = testN[t];
97
98         /* Allocate array and fill with 0 to N-1 */
99         int *a = (int *)malloc(N * sizeof(int));
100        for (int i = 0; i < N; i++) {
101            a[i] = i; /* sorted array: 0, 1, 2, ..., N-1 */
102        }
103        int target = N; /* worst case: N is not in the array */
104
105        printf("\n===== N = %d =====\n", N);
106
107        clock_t start, stop; /* timing variables */
108        int K; /* repetition count */
109        long ticks; /* elapsed ticks */
110
111        /* --- Binary Search Iterative --- */
112        K = 1;
113        do {
114            K *= 2; /* double K each attempt */
115            start = clock();
116            for (int i = 0; i < K; i++) {
117                BinarySearch_Iterative(a, N, target); /* call function K times
118                */
119            }
119            stop = clock();
120        } while (stop - start < 10); /* ensure ticks >= 10 */
121        ticks = stop - start;
122        printResult("Binary Search (iterative)", K, ticks);
123
124        /* --- Binary Search Recursive --- */

```

```

125     K = 1;
126     do {
127         K *= 2;
128         start = clock();
129         for (int i = 0; i < K; i++) {
130             BinarySearch_Recursive(a, target, 0, N - 1); /* pass left=0,
131                                                         right=N-1 */
132         }
133         stop = clock();
134         ticks = stop - start;
135         printResult("Binary Search (recursive)", K, ticks);
136
137         /* --- Sequential Search Iterative --- */
138         K = 1;
139         do {
140             K *= 2;
141             start = clock();
142             for (int i = 0; i < K; i++) {
143                 SequentialSearch_Iterative(a, N, target);
144             }
145             stop = clock();
146             ticks = stop - start;
147             printResult("Sequential Search (iterative)", K, ticks);
148
149             /* --- Sequential Search Recursive --- */
150             K = 1;
151             do {
152                 K *= 2;
153                 start = clock();
154                 for (int i = 0; i < K; i++) {
155                     SequentialSearch_Recursive(a, N, target, 0); /* start from
156                                                                     index 0 */
157                 }
158                 stop = clock();
159                 ticks = stop - start;
160                 printResult("Sequential Search (recursive)", K, ticks);
161
162                 free(a); /* release allocated memory */
163             }
164             return 0;
165         }
166     }

```

Declaration

I hereby declare that all the work done in this project titled “Searching Algorithms Performance Analysis” is of my independent effort.